

```
[1]: # Program to find squares using map()
numbers = [1, 2, 3, 4, 5]

squares = list(map(lambda x: x * x, numbers))

print("Original List:", numbers)
print("Squares List:", squares)
```

Original List: [1, 2, 3, 4, 5]

Squares List: [1, 4, 9, 16, 25]

```
[2]: # Program to filter even numbers

numbers = [10, 15, 20, 25, 30, 35, 40]

even_numbers = list(filter(lambda x: x % 2 == 0, numbers))

print("Original List:", numbers)
print("Even Numbers:", even_numbers)
```

Original List: [10, 15, 20, 25, 30, 35, 40]

Even Numbers: [10, 20, 30, 40]

```
[3]: # Program to find sum using reduce()

from functools import reduce

numbers = [1, 2, 3, 4, 5]

sum_result = reduce(lambda x, y: x + y, numbers)

print("List Elements:", numbers)
print("Sum of Elements:", sum_result)
```

List Elements: [1, 2, 3, 4, 5]

Sum of Elements: 15

[4]: # Program demonstrating higher-order function

```
def multiplier(n):  
    return lambda x: x * n
```

```
double = multiplier(2)
```

```
triple = multiplier(3)
```

```
print("Double of 5:", double(5))
```

```
print("Triple of 5:", triple(5))
```

Double of 5: 10

Triple of 5: 15

[5]: # Program to sort tuples using lambda

```
students = [  
    ("Anu", 85),  
    ("Rahul", 72),  
    ("Priya", 90),  
    ("Kiran", 78)  
]
```

```
sorted_students = sorted(students, key=lambda x: x[1])
```

```
print("Students Sorted by Marks:")
```

```
for student in sorted_students:  
    print(student)
```

Students Sorted by Marks:

('Rahul', 72)

('Kiran', 78)

('Anu', 85)

('Priya', 90)

[6]: # Program to convert strings to uppercase using map()

```
names = ["anu", "rahul", "priya", "kiran"]
```

Students Sorted by Marks:

```
('Rahul', 72)
('Kiran', 78)
('Anu', 85)
('Priya', 90)
```

[6]: # Program to convert strings to uppercase using map()

```
names = ["anu", "rahul", "priya", "kiran"]

uppercase_names = list(map(lambda x: x.upper(), names))

print("Original Names:", names)
print("Uppercase Names:", uppercase_names)
```

```
Original Names: ['anu', 'rahul', 'priya', 'kiran']
Uppercase Names: ['ANU', 'RAHUL', 'PRIYA', 'KIRAN']
```

[7]: # Program to filter positive numbers

```
numbers = [-10, 20, -30, 40, 50, -60]

positive_numbers = list(filter(lambda x: x > 0, numbers))

print("Original List:", numbers)
print("Positive Numbers:", positive_numbers)
```

```
Original List: [-10, 20, -30, 40, 50, -60]
Positive Numbers: [20, 40, 50]
```

[8]: # Program to find maximum number using reduce()

```
from functools import reduce

numbers = [12, 45, 67, 23, 89, 34]

maximum = reduce(lambda x, y: x if x > y else y, numbers)
```

```
[8]: # Program to find maximum number using reduce()

from functools import reduce

numbers = [12, 45, 67, 23, 89, 34]

maximum = reduce(lambda x, y: x if x > y else y, numbers)

print("List Elements:", numbers)
print("Maximum Element:", maximum)
```

```
List Elements: [12, 45, 67, 23, 89, 34]
Maximum Element: 89
```

```
[9]: # Program using lambda function for addition

addition = lambda a, b: a + b

num1 = 10
num2 = 20

result = addition(num1, num2)

print("First Number:", num1)
print("Second Number:", num2)
print("Sum:", result)
```

```
First Number: 10
Second Number: 20
Sum: 30
```

```
[10]: # Program to calculate length of strings using map()

words = ["Python", "Java", "C", "JavaScript"]

lengths = list(map(lambda x: len(x), words))

print("Words:", words)
```

```
List Elements: [12, 45, 67, 23, 89, 34]
Maximum Element: 89
```

```
[9]: # Program using lambda function for addition
```

```
addition = lambda a, b: a + b
```

```
num1 = 10
```

```
num2 = 20
```

```
result = addition(num1, num2)
```

```
print("First Number:", num1)
```

```
print("Second Number:", num2)
```

```
print("Sum:", result)
```

```
First Number: 10
```

```
Second Number: 20
```

```
Sum: 30
```

```
[10]: # Program to calculate length of strings using map()
```

```
words = ["Python", "Java", "C", "JavaScript"]
```

```
lengths = list(map(lambda x: len(x), words))
```

```
print("Words:", words)
```

```
print("Lengths:", lengths)
```

```
Words: ['Python', 'Java', 'C', 'JavaScript']
```

```
Lengths: [6, 4, 1, 10]
```